

MANUAL TÉCNICO DEL PROGRAMADOR

Proyecto: Modelado de Datos con Estructuras (STRUCT) y Punteros — Actividad 5
Institución: E.E.S.T. N° 1 “Escuela de Educación Técnica N.° 1 "Eduardo Ader" ” – Vicente López.
Asignatura: Laboratorio de Programación (LPR) — 5° Año
Docente: Ganduglia Yamil y Mansilla Muñoz York Elías
Autor / Grupo: Mikaela Batirola
Fecha: 7 de octubre de 2026 | **Versión:** v1.0.0

CONTROL DE VERSIONES Y CHANGELOG

Versión	Fecha	Autor	Descripción de Cambios
v1.0.0	07/10/2026	Mikaela Batirola	Versión inicial del programa de modelado de datos mediante estructuras y punteros en C++

1. ARQUITECTURA DEL SISTEMA Y ENTORNO DE DESARROLLO

Requisitos del Entorno

- **Lenguaje:** C++.
- **Compilador:** g++.
- **IDE recomendado:** Visual Studio Code con extensión C/C++.
- **Shell:** Windows PowerShell.

Estructura de Directorios del Repositorio

sistemaproyecto/

```
|— .gitignore          <-- Reglas de exclusión (.exe, .vscode/)
|— LICENSE             <-- Licencia MIT escolar
|— README.md           <-- Portada con integrantes y descripción del modelo
|— docs/               <-- Carpeta de documentación formal
|   |— InformeEEST1_LPR2026_ACT05_G03_v1.0.0.pdf <-- ENTREGABLE EN PDF
|   (APA v7)
|       |— manuales/
|           |— manual_programador_v1.0.0.pdf <-- Explicación del struct y operador ->
|           |— manual_programador_v1.0.0.md <-- Explicación del struct y operador ->
|— src/                <-- Código fuente C++
|   |— main.cpp         <-- Programa principal modular
|— capturas/           <-- Evidencias de ejecución (.png)
|   |— ejecucion_struct.png
```

2. DOCUMENTACIÓN DE ESTRUCTURAS, PUNTEROS Y FUNCIONES

Módulo 1: struct EntidadProyecto

- **Propósito:** Agrupar diferentes datos relacionados dentro de una misma entidad.
- **Definición:**

```
struct EntidadProyecto {

    int id;

    char nombre[50];

    float metrica;

};
```

- **Miembros:**

- `id`: identificador numérico de la entidad.
- `nombre`: almacena el nombre o descripción.
- `metrica`: almacena un valor decimal relacionado con la operación.

El uso de `struct` permite organizar diferentes tipos de datos dentro de una única estructura, evitando trabajar con variables independientes para cada dato.

Módulo 2: Inicialización de `EntidadProyecto`

- **Propósito:** Crear la entidad e inicializar sus valores antes del ingreso de datos.
- **Declaración:**

```
EntidadProyecto miEntidad = {0, "Mikaela Batirola", 0.0f};
```

- **Valores iniciales:**
 - `id = 0`
 - `nombre = "Mikaela Batirola"`
 - `metrica = 0.0f`

La inicialización permite que la estructura tenga valores definidos antes de ser modificada por la función de carga.

Módulo 3: `void cargarDatos(EntidadProyecto* ptr)`

- **Propósito:** Cargar y modificar los datos de la estructura mediante un puntero.
- **Firma:**

```
void cargarDatos(EntidadProyecto* ptr);
```

- **Mecánica:** La función recibe la dirección de memoria de `miEntidad` y utiliza el puntero `ptr` para acceder directamente a sus miembros.

La llamada se realiza mediante:

```
cargarDatos(&miEntidad);
```

El operador `&` obtiene la dirección de memoria de la variable.

Módulo 4: Operador Flecha `->`

- **Propósito:** Acceder a los miembros de una estructura cuando se trabaja con un puntero.
- **Ejemplos:**

```
ptr->id;
```

```
ptr->nombre;
```

```
ptr->metrica;
```

El operador `->` permite acceder directamente a los miembros de la estructura apuntada por `ptr`.

La diferencia con el operador punto es:

```
miEntidad.id;
```

cuando se trabaja directamente con la estructura, y:

```
ptr->id;
```

cuando se trabaja mediante un puntero.

Módulo 5: Ingreso de datos mediante punteros

El ID se ingresa mediante:

```
cin >> ptr->id;
```

Luego se utiliza:

```
cin.ignore();
```

para limpiar el carácter de salto de línea que queda en el buffer después de utilizar `cin`.

A continuación, se utiliza:

```
cin.getline(ptr->nombre, 50);
```

para ingresar el nombre o descripción.

Finalmente, la métrica se carga mediante:

```
cin >> ptr->metrica;
```

De esta manera, los tres miembros de la estructura son modificados directamente mediante el puntero.

Módulo 6: Verificación y dirección de memoria

Una vez finalizada la carga de datos, el programa vuelve a `main()` y muestra la información almacenada:

```
cout << "ID Registrado: " << miEntidad.id << endl;  
cout << "Nombre Registrado: " << miEntidad.nombre << endl;  
cout << "Metrica Guardada: " << miEntidad.metrica << endl;
```

También se obtiene la dirección de memoria de la estructura mediante:

```
cout << "Direccion RAM Hexadecimal: " << &miEntidad << endl;
```

Esto permite verificar los datos almacenados y observar la dirección de memoria utilizada por la entidad.

3. INSTRUCCIONES DE COMPILACIÓN Y EJECUCIÓN

Compilación desde PowerShell

Ubicándose en la raíz del proyecto, ejecutar:

```
g++ src/main.cpp -o src/sistemaproyecto.exe
```

Ejecución del programa

```
.\src\sistemaproyecto.exe
```

Diagnóstico de Errores Frecuentes

1. **Error: 'g++' no se reconoce como un comando...**
 - **Causa:** El compilador g++ no está configurado correctamente en las variables de entorno PATH.
 - **Solución:** Verificar la instalación de MinGW-w64 y configurar correctamente su carpeta bin en el PATH.
2. **Error al ingresar el nombre o descripción**
 - **Causa:** No utilizar `cin.ignore()` antes de `getline()`.
 - **Solución:** Colocar `cin.ignore()` después del ingreso del ID y antes de `cin.getline()`.
3. **Error al acceder a los miembros mediante el puntero**
 - **Causa:** Utilizar el operador punto (.) en lugar del operador flecha (->).
 - **Solución:** Cuando la variable es un puntero, utilizar `ptr->miembro`.

4. REFERENCIAS

- Documentación y material de clase de Laboratorio de Programación (LPR), Actividad 5, 2026.